FFT and Pattern Matching Algorithms

CSC 821 - Group 1 | Module 9 | Author: Enyoni

> **Objective.** Implement the Fast Fourier Transform from scratch, use it to identify frequency components in a test signal, and implement and compare Naive Search, Knuth-Morris-Pratt (KMP), and Boyer-Moore exact pattern matching.

1. Fast Fourier Transform

The Discrete Fourier Transform (DFT) converts a sequence of time-domain samples into frequency-domain coefficients. A direct DFT calculates each of the N output coefficients from all N samples, giving a time complexity of $O(N^2)$. The radix-2 Cooley-Tukey FFT exploits symmetry in the complex roots of unity. It recursively splits the input into even-indexed and odd-indexed samples, transforms both halves, and combines them using twiddle factors.

The recurrence reduces the running time to O(N log N), provided the sample count is a power of two. The notebook implementation validates this requirement, handles the one-sample base case, and performs the recursive transform using Python complex arithmetic. NumPy is used to construct the signal, arrange the results, and validate the custom transform, but `numpy.fft` is not used to produce the experimental spectrum.

Signal experiment

The one-second test signal is sampled at 256 Hz and combines a 20 Hz sine wave of amplitude 1.0 with a 50 Hz sine wave of amplitude 0.5. The time-domain plot shows the combined oscillation. In the frequency domain, the custom FFT separates the signal into distinct peaks at 20 Hz and 50 Hz, with peak magnitudes reflecting the original amplitudes.

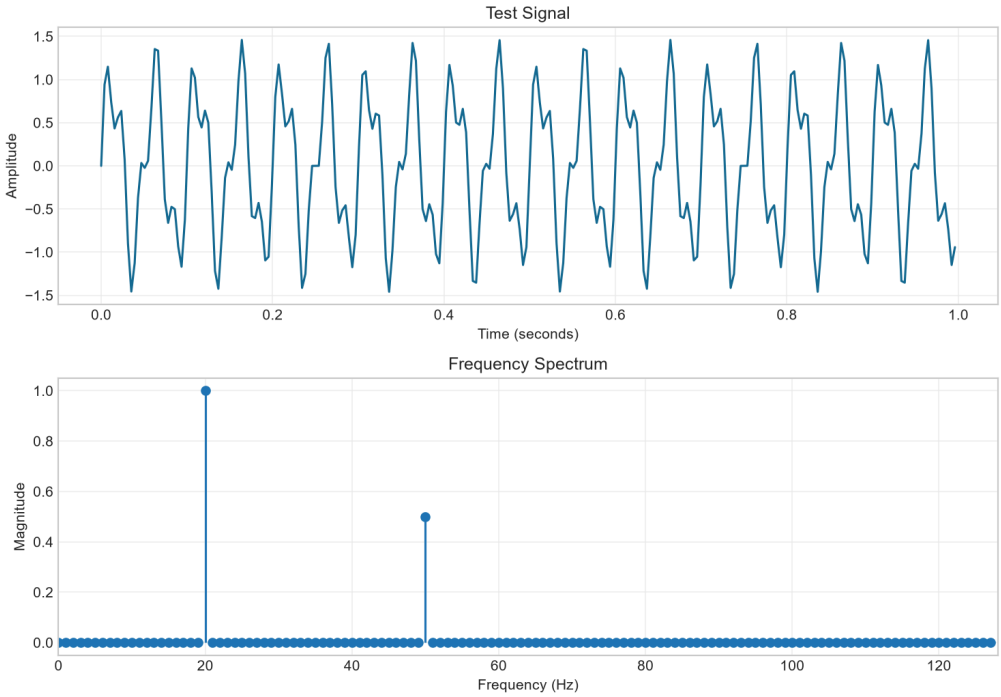

Figure 1. Combined test signal and its one-sided FFT magnitude spectrum.

FFT is important because it makes frequency analysis, filtering, fast convolution, compression, audio processing, image processing, and digital communications computationally practical. Validation against `numpy.fft.fft` confirmed that the custom coefficients were numerically equivalent for the test input.

Pattern Matching Analysis

CSC 821 - Group 1 | Module 9 Report

2. Exact Pattern Matching

The three algorithms search for every occurrence of a pattern in a text and return the starting indices, including overlapping matches. For the sample text `ABAAABCDABCABC` and pattern `ABC`, all methods correctly return `[4, 8, 11]`.

Naive Search

Naive Search aligns the pattern at every possible text position and compares characters from left to right. It has no preprocessing cost and is easy to understand. Its worst-case running time is O(nm), where n is the text length and m is the pattern length. It is suitable for short inputs and one-off searches where setup overhead matters more than scalability.

Knuth-Morris-Pratt

KMP preprocesses the pattern into a longest-proper-prefix/suffix (LPS) table. Following a mismatch, the table identifies how much of the matched prefix can be reused, so the text index never moves backward. Preprocessing costs O(m), searching costs O(n), and total worst-case time is O(n + m). KMP is preferred for repetitive patterns and applications that require predictable linear performance.

Boyer-Moore

Boyer-Moore compares the pattern from right to left. This implementation uses the bad-character rule: after a mismatch, the last occurrence of that text character in the pattern determines how far the pattern can shift. Preprocessing costs O(m). Although this bad-character-only version has an O(nm) worst case, it is often sublinear in practice because one comparison may skip several alignments. It is effective for long patterns and large alphabets such as natural-language text.

Algorithm	Preprocessing	Search complexity	Preferred use
Naive	O(1)	O(nm) worst case	Short text, one-off search, simplest implementation
KMP	O(m)	O(n)	Repeated prefixes and guaranteed linear searching
Boyer-Moore	O(m)	Often sublinear; O(nm) worst case here	Long patterns and large alphabets

3. Findings and Experience

- The custom FFT recovered the expected 20 Hz and 50 Hz frequency components.
- All three matching algorithms produced identical results for normal, overlapping, absent, and empty-pattern test cases.
- Preprocessing can avoid repeated work: KMP reuses prefix information, while Boyer-Moore skips impossible alignments.
- Algorithm choice depends on input size, pattern structure, alphabet size, and whether worst-case guarantees are required.

The exercise demonstrates a common algorithm-design principle: exploiting mathematical or structural properties can preserve correctness while reducing computational effort. FFT uses symmetry to improve on direct DFT computation, while KMP and Boyer-Moore use information about the pattern to avoid unnecessary character comparisons.